

Outline Introduction to Autonomous Vehicles

- Perception
 - Machine Learning Workflow
 - Sensor and Camera Calibration
 - Neuronal Networks
 - Image Classification
 - 2D Object detection
- Sensor Fusion
 - Sensor Fusion and Perception
 - Lidar Sensor
 - 3D Object Detection
 - Kalman Filters and Extended Kalman filters
 - Multi Object Tracking
- Localization
 - Markov Localization
 - Scan Matching Algorithms 2D and 3D
- **Decision Making and Planning**
 - Behavioral Planning
 - **Trajectory Generation**
 - **Motion Planning**
- Control
 - Proportional-Integral-Derivative (PID) Control
 - Model Predictive Control (MPC)
- Human Factors

Trajectory Planning

Overview:

Definition of the Motion Planning Problem

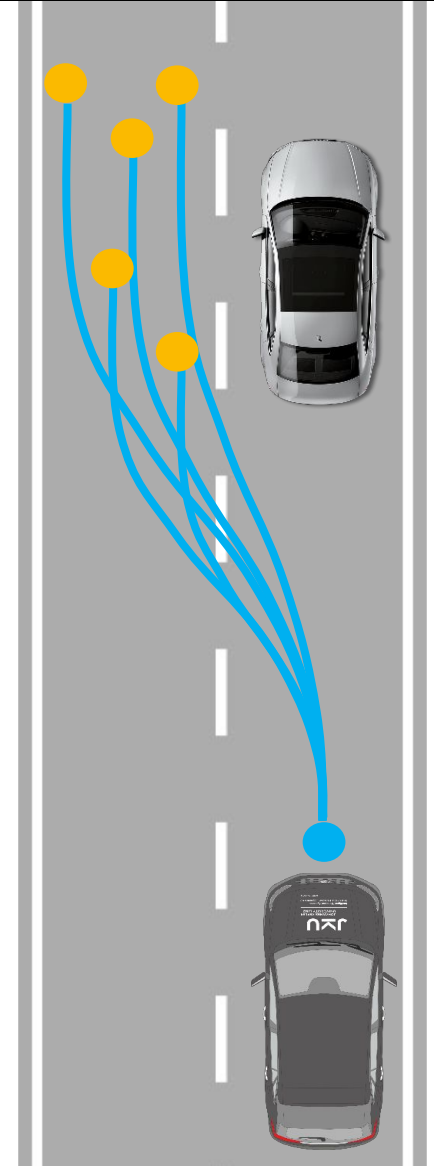
Introduction of the types of Planning Approaches

Introduction of A* (A Star)

Introduction of Hybrid A*

Continuity, Optimization, Constraints

Sampling Based Polynomial Trajectory Generation



The Motion Planning Problem

Configuration Space:

Defines all possible configurations of our car

The two/three-dimensional world configuration space can be...

- 2D[x,y] or...
- 3D[x,y,theta]

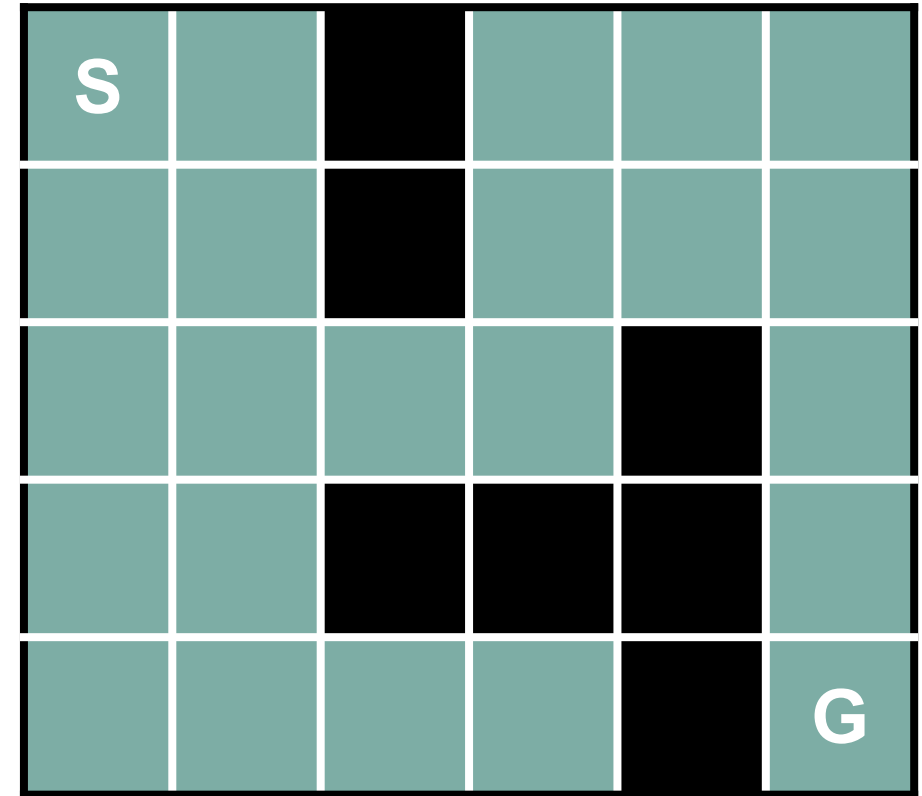
Given:

A start configuration q_{start} (from localization and sensors)

A goal configuration q_{goal} (from behavior layer)

Constraints (physics, map, traffic)

Problem:



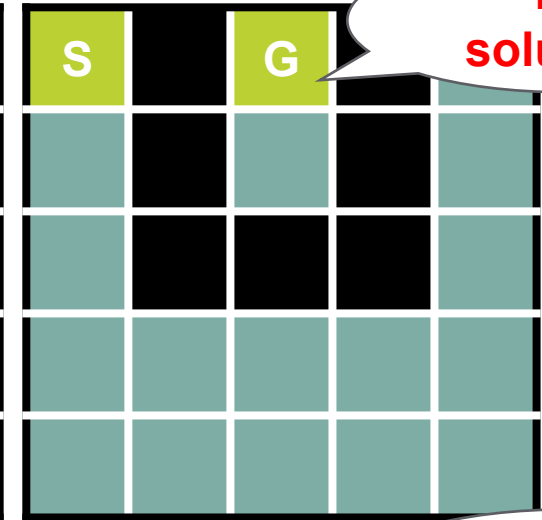
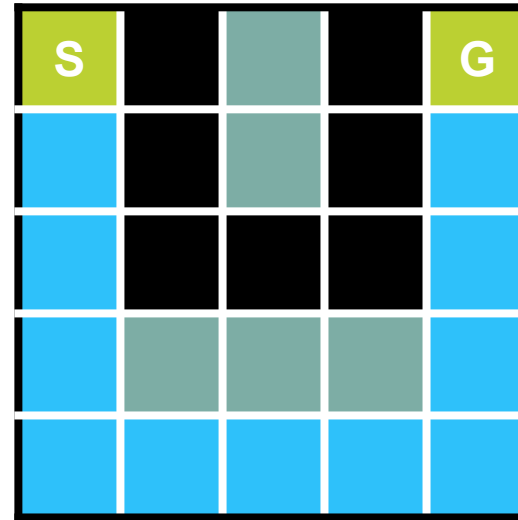
Motion Planning Algorithms Properties

Completeness:

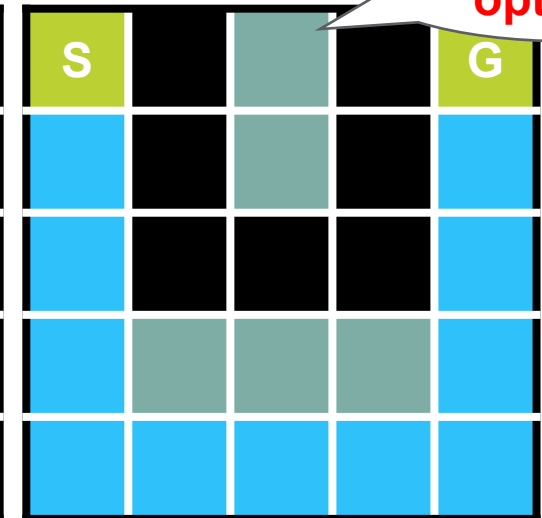
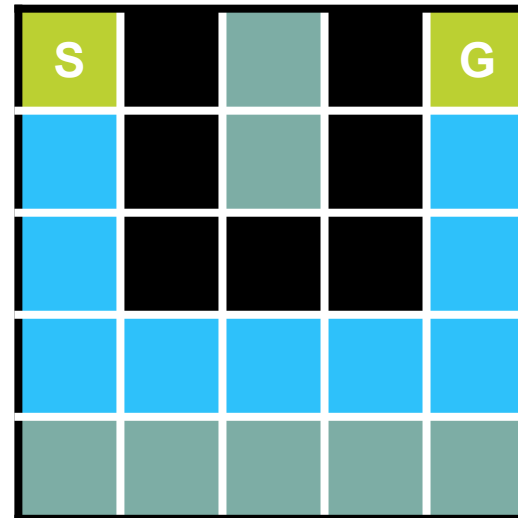
- if a solution exists, planner always finds it
- if no solution exists, planner terminates and reports failure

Optimality:

- given a cost function for evaluating a sequence of actions, planner always returns a feasible sequence of actions with minimal cost



No
solution!



Not
optimal!

Motion Planning Algorithms

Combinatorial Methods

Construct a graph that represents the car's environment, and search for a path from the starting position to the goal position

**Efficient, simple
but struggle with complex environments**

Potential Field Methods

Car as point in the continuous space, potential field that represents the potential energy of the car at each point in space. Car moves towards areas of lower potential energy until it reaches the goal.

**Simple, effective
but can get stuck in local minima**

Optimal Control Methods

Use optimization techniques to find the best possible trajectory

**Can be very precise
but can be computationally expensive and require accurate models of the robot and environment**

Sampling Based Methods

They randomly sample the car's configuration space. Samples to build a roadmap of the environment. Roadmap to plan a path from starting to goal position

**Can handle complex environments, Very efficient
But can struggle with narrow passages and require large number of samples for accuracy**

Sampling Based Methods

Discrete methods: Discretization of the configuration and input space

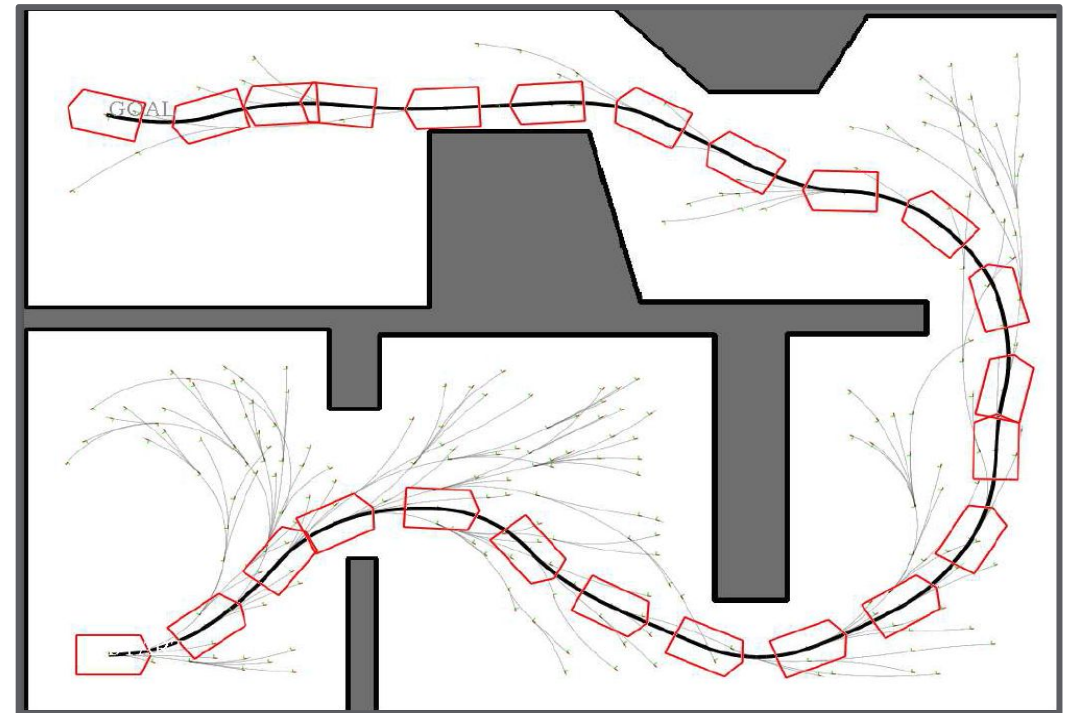
□ **Deterministic graph search algorithms:**

- A*, D*, D*-lite, Dijkstra's, ARA*, etc.

7	6	5	6	7	8	9	10	11		19	20	21	22
6	5	4	5	6	7	8	9	10		18	19	20	21
5	4	3	4	5	6	7	8	9		17	18	19	20
4	3	2	3	4	5	6	7	8		16	17	18	19
3	2	1	2	3	4	5	6	7		15	16	17	18
2	1	0	1	2	3	4	5	6		14	15	16	17
3	2	1	2	3	4	5	6	7		13	14	15	16
4	3	2	3	4	5	6	7	8		12	13	14	15
5	4	3	4	5	6	7	8	9	10	11	12	13	14
6	5	4	5	6	7	8	9	10	11	12	13	14	15

Probabilistic methods: Random exploration of the configuration and input space

- RRT, RRT*, PRM, etc.



Introduction to A*

A* is a **heuristic search** algorithm that finds the shortest path between two points in a graph, considering the **cost of each path** and an **estimate of the distance remaining** to the goal.

G score: Path Cost from the Start Node

H score: Heuristic Cost to reach the Goal Node

F score: Total Cost = $G + H$

Maintaining two lists of nodes:

Open List - Nodes queued up for searching

Closed List - Nodes that have already been searched

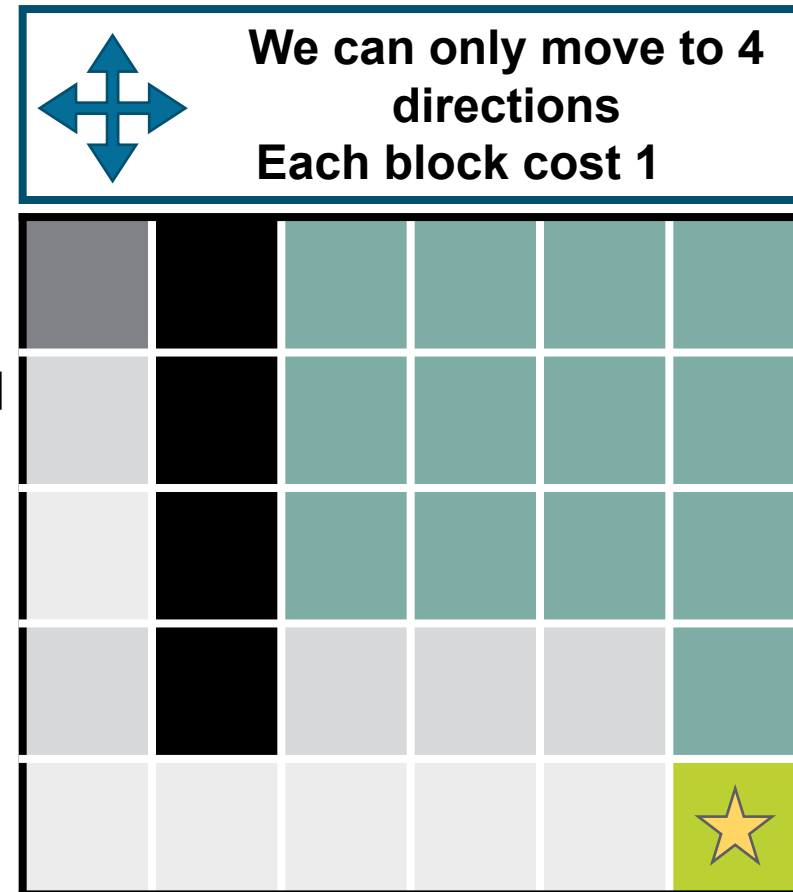
Keep going until

Current Node == Goal Node

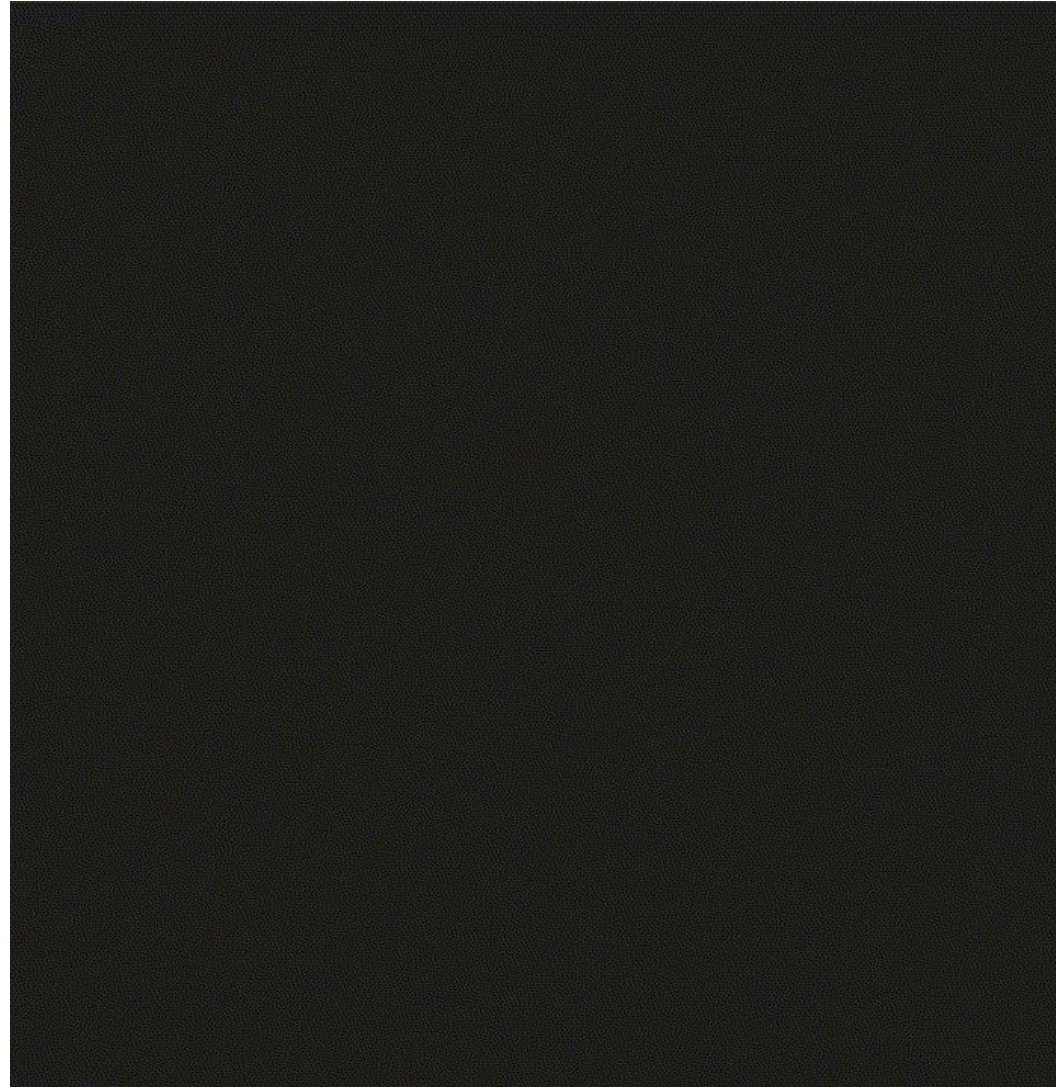
Or

Open List is Empty (**no solution**)

NOTE: A* is a **DISCRETE** method



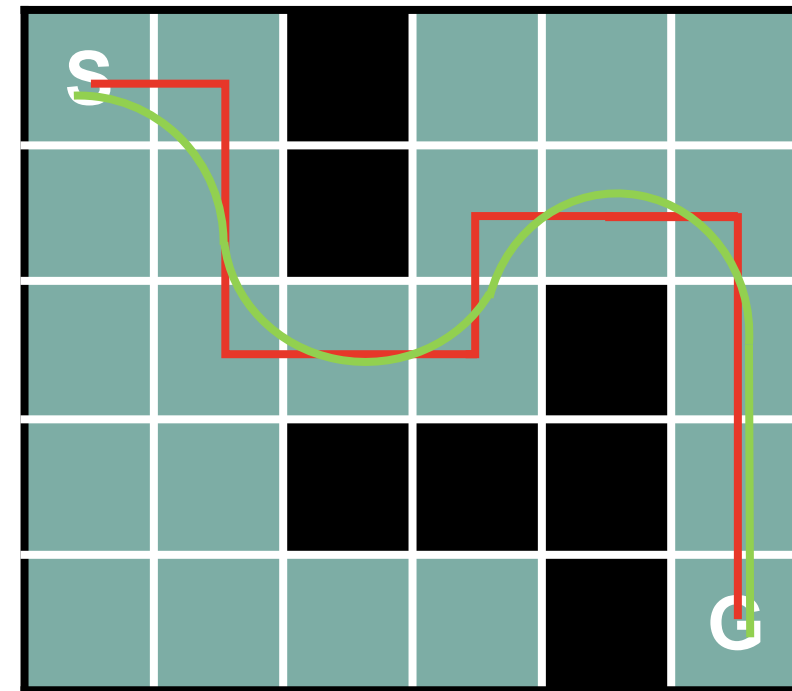
A-star algorithm



Introduction to Hybrid A*

Hybrid A* is an extension of the A* for **non-holonomic systems**, it **combines discrete search and continuous search**:

1. Discretizing the continuous space into a grid of cells and creating a graph where each cell is a node.
2. Apply A* to this graph to find an approximate path (gives you the holonomic heuristic cost).
3. Refine the approximate path using a continuous search algorithm (gives you the non-holonomic cost).
4. Find a more accurate and feasible path for the non-holonomic system.



A*
is limited to finding paths in a
grid-like environment



Hybrid A*
handles continuous environments
with non-zero turning radius

Hybrid A* Heuristics

Nonholonomic-without-obstacles heuristic:

Ignores obstacles but takes into account the nonholonomic nature of the car, and can be completely precomputed for vehicle location, orientation, and direction of motion, which helps in approaching the goal with the desired heading.

Holonomic-with-obstacles heuristic

Ignores the nonholonomic nature of the car but computes the shortest distance to the goal. It is calculated online by performing dynamic programming in 2-D (ignoring vehicle orientation and motion direction).

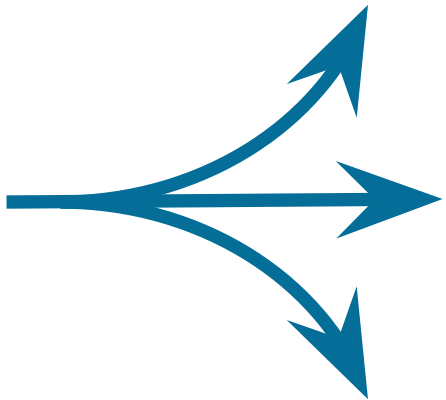
Both heuristics are admissible, so the maximum of the two can be used.

For more details read **Section 6.3 - Free Form Navigation** of the paper **Junior: The Stanford Entry in the Urban Challenge**: <https://onlinelibrary.wiley.com/doi/pdfdirect/10.1002/rob.20258>

Hybrid A* in Practice

PRACTICAL CONSIDERATIONS

- Use maximum steering angle in both directions and 0.
 - -35 deg, 0 deg, 35 deg
- For given speed, we assemble path out of 3 components:



we use bicycle model:

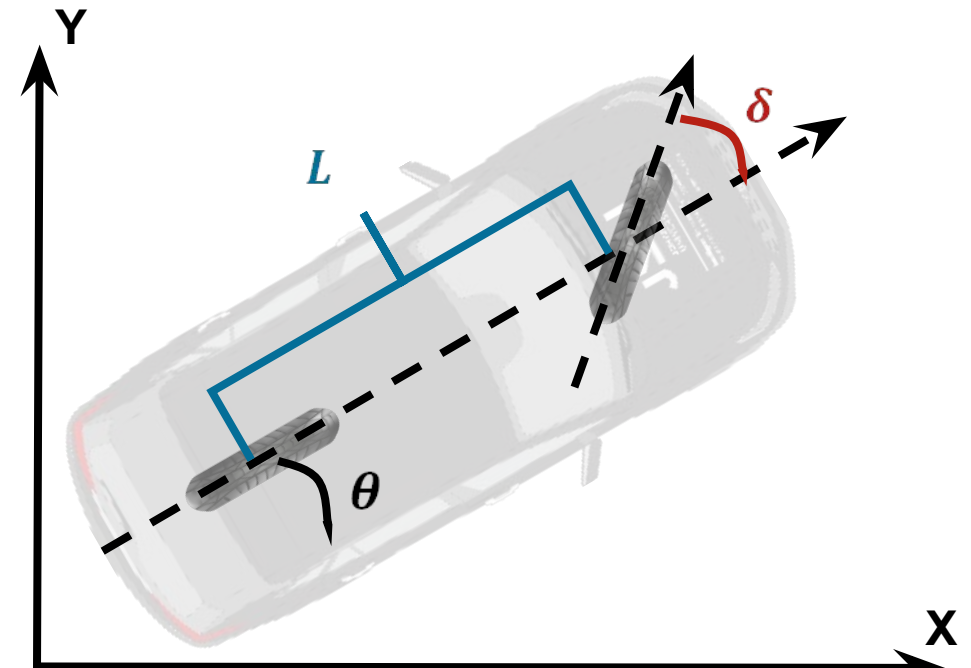
θ **theta** is the heading direction,
 ω **omega** is the heading rate of change
 L is the distance between front and rear axle
 δ **delta** is the steering angle
 v is the constant positive velocity

$$x(t + \Delta t) = x(t) + v\Delta t \cos(\theta)$$

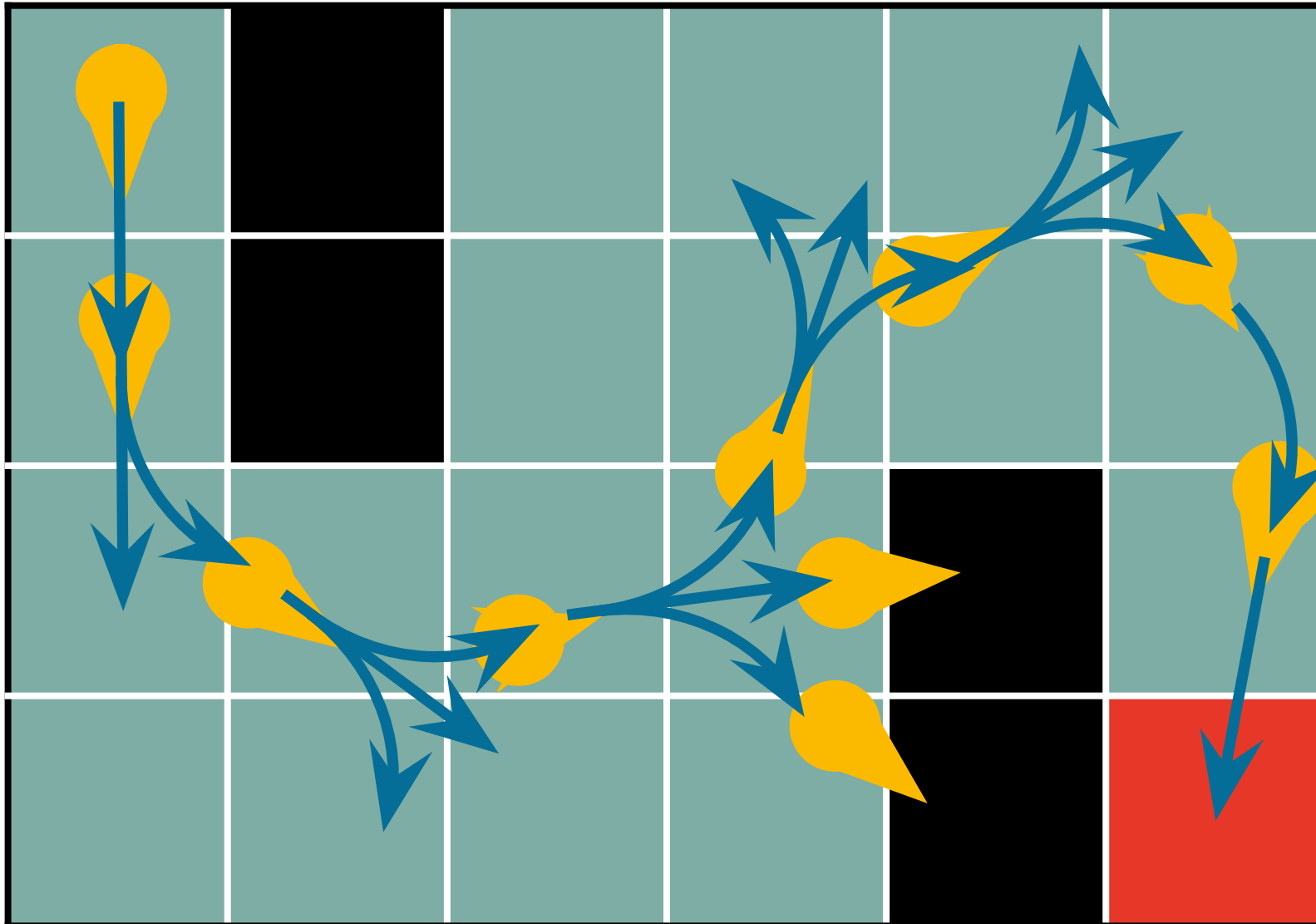
$$y(t + \Delta t) = y(t) + v\Delta t \sin(\theta)$$

$$\theta(t + \Delta t) = \theta(t) + \omega\Delta t$$

$$\omega = \frac{v}{L} \tan(\delta(t))$$



Hybrid A* in Practice



Implementing Hybrid A*

Introduction:

In this exercise, you will be provided a working implementation of a **breadth first search** algorithm which does **not** use any heuristics to improve its efficiency. Your goal is to try to make the appropriate modifications to the algorithm so that it takes advantage of heuristic functions to reduce the number of grid cell expansions required.

Note: The exercise does not have a solution, it is just for practice.

Instructions:

Modify the code and run the Jupyter notebook to check your results. Note the number of expansions required to solve an empty 15x15 grid (it should be about 18,000!). Modify the code to try to reduce that number. How small can you get it?

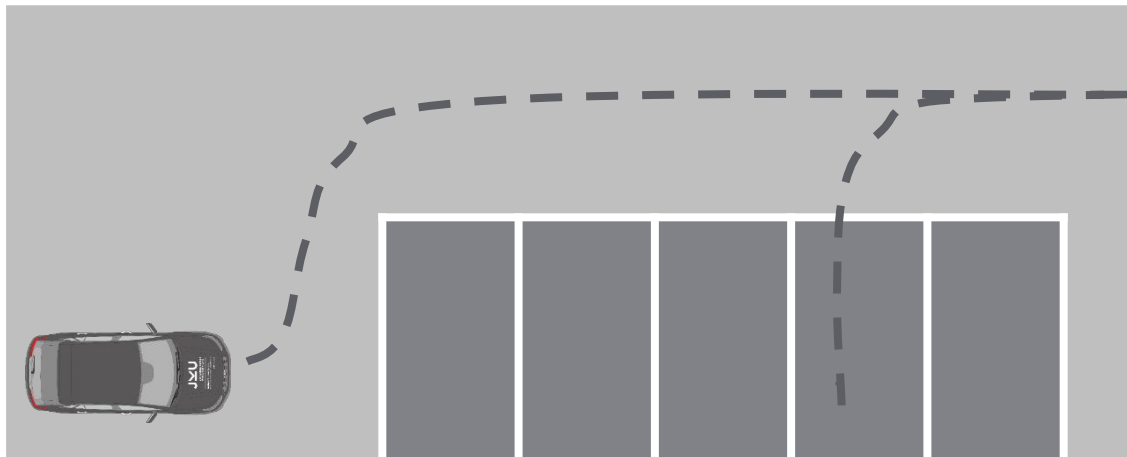
Code: See EX Planning-20230423T175220Z-001.zip in Moodle

Environment Classification

Unstructured:

Example: Parking lot (maze)

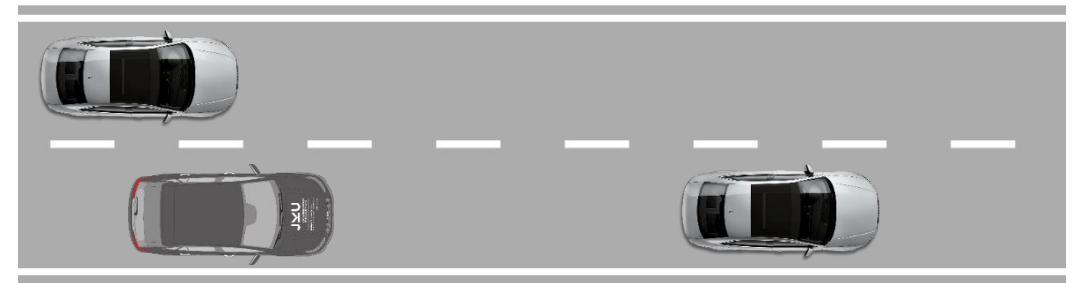
- Less specific rules and lower speeds
- No obvious reference path or trajectory



Structured:

Example: Highway, Street Driving

- Predefined rules regarding how to move on the road
 - Direction of traffic
 - Lane boundaries
 - Speed limits
- Road structure can be used as reference

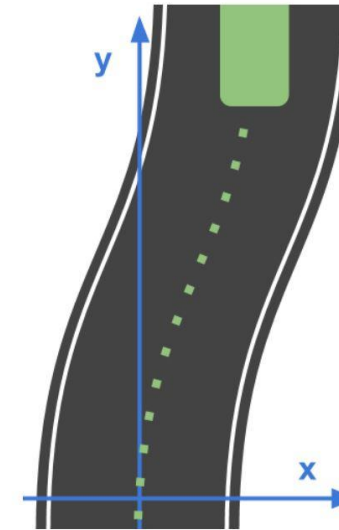


Cartesian vs. Frenet Coordinates

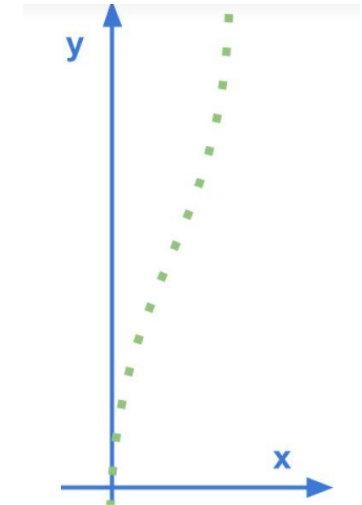
Cartesian

usually used to describe the position of objects, but it is not the best choice for vehicles.

- difficult to know where the road is
- difficult to determine if vehicle is off the center of the lane



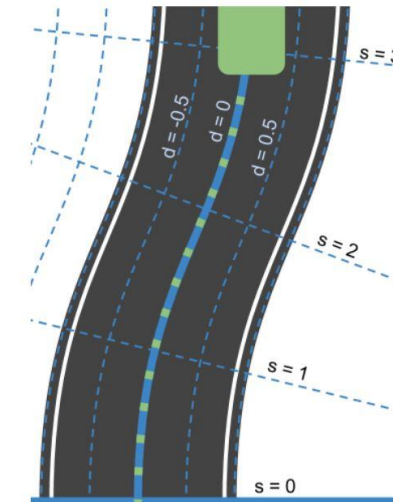
Cartesian



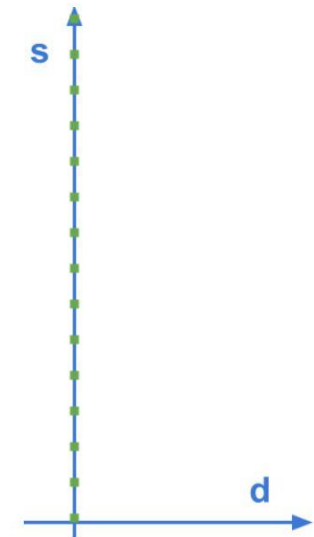
Frenet

An alternative solution to the Cartesian coordinate system, which describes the position of the car relative to the road.

- Ordinate: s direction, represents the distance along the road
- Abscissa: d direction, represents the displacement from the longitudinal line



Frenet



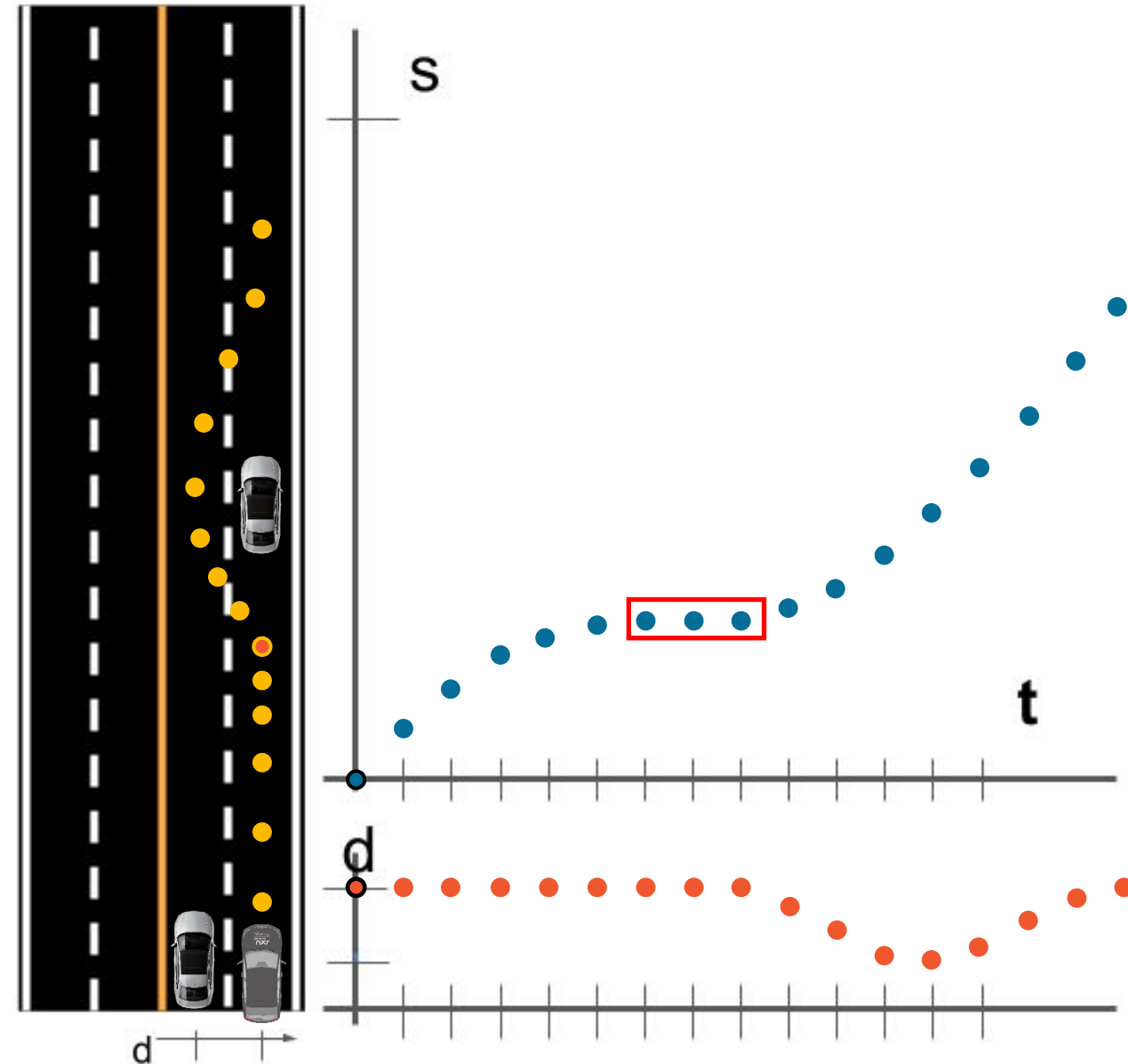
The Need for Time

Problems with s and d but no t :

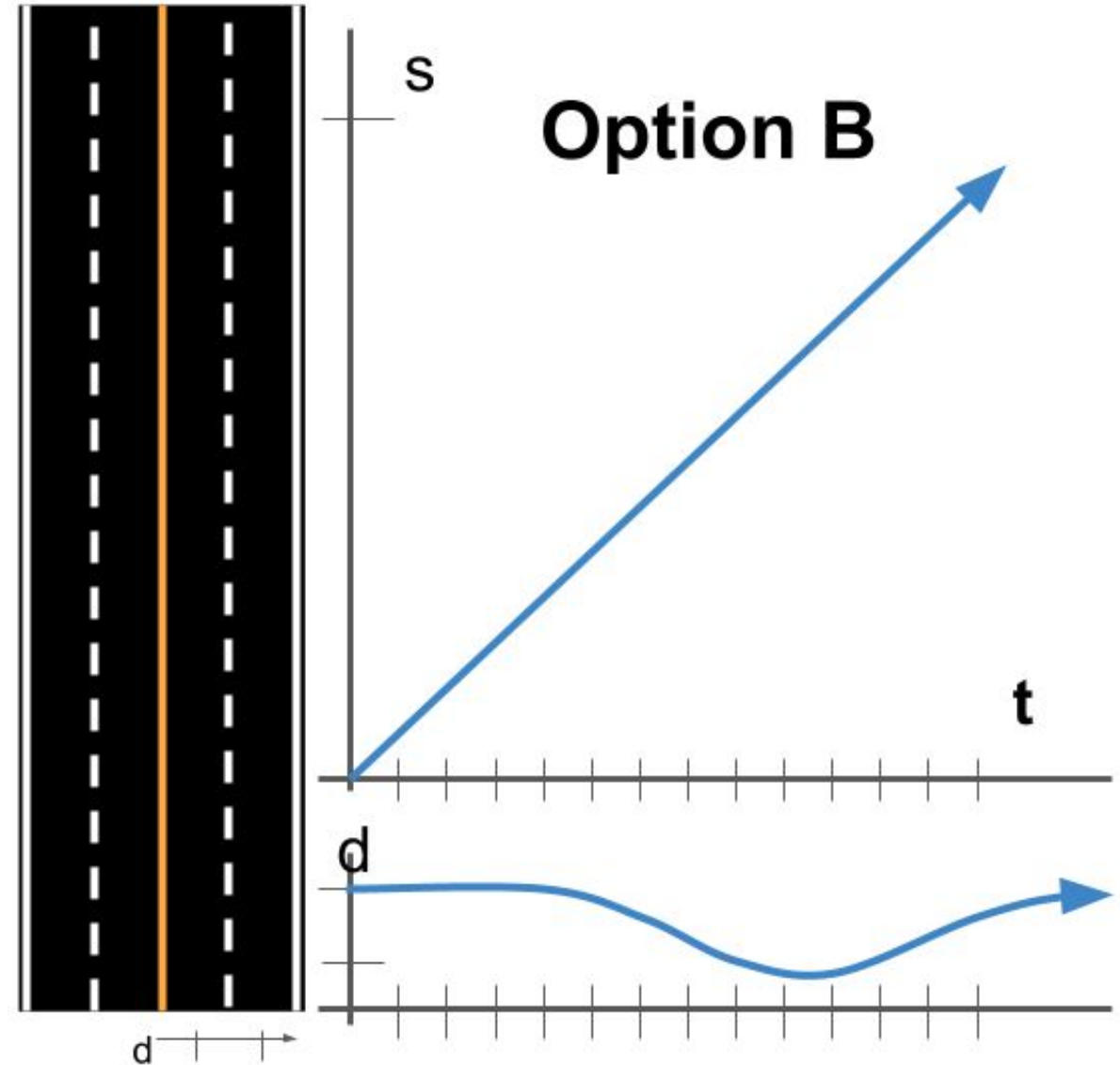
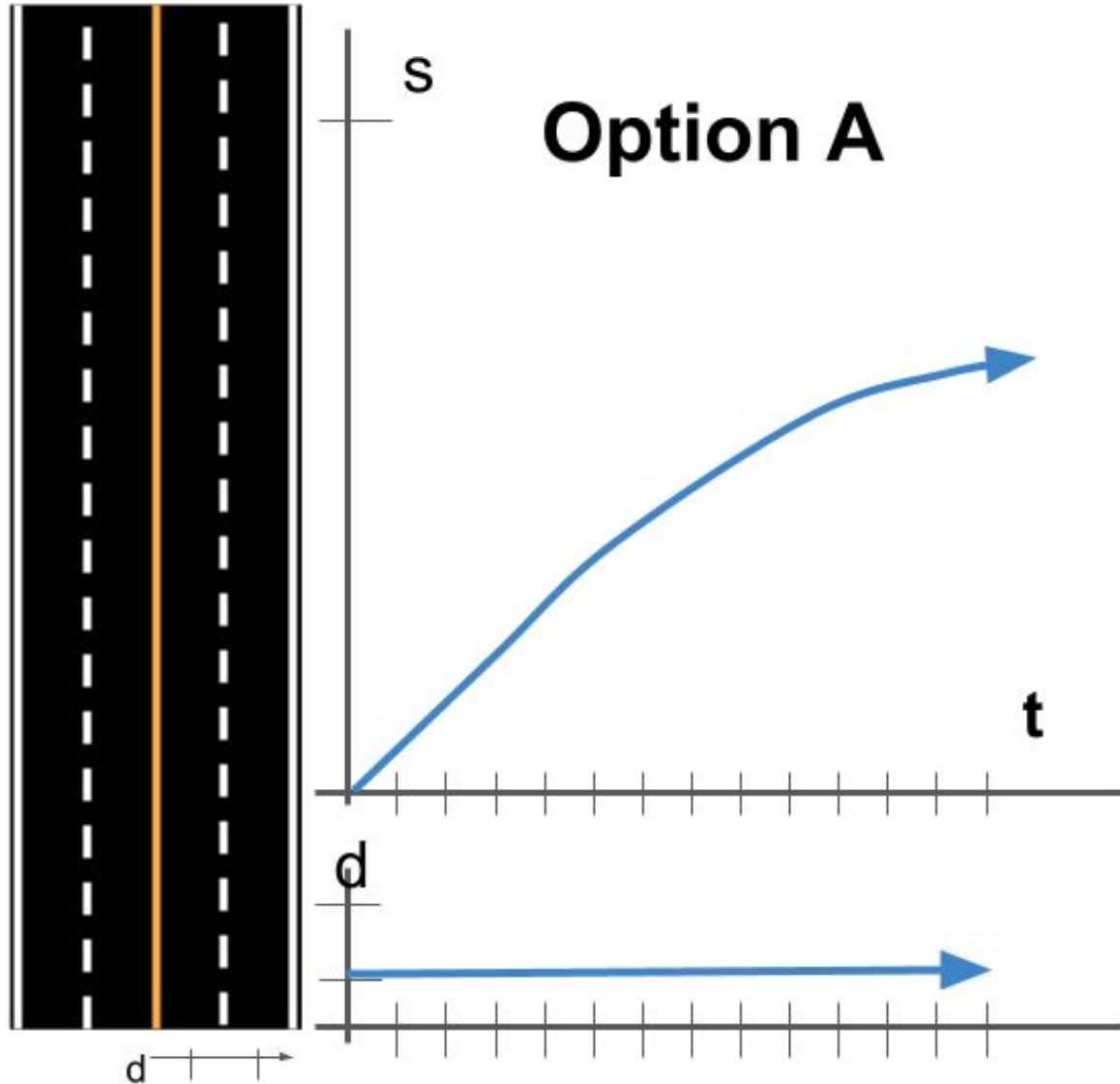
- A path that does not consider time t is not accurate
- Path Planning without considering time does not take predictions into account

Path: a sequence of configurations (s, d , and *sometimes* θ)

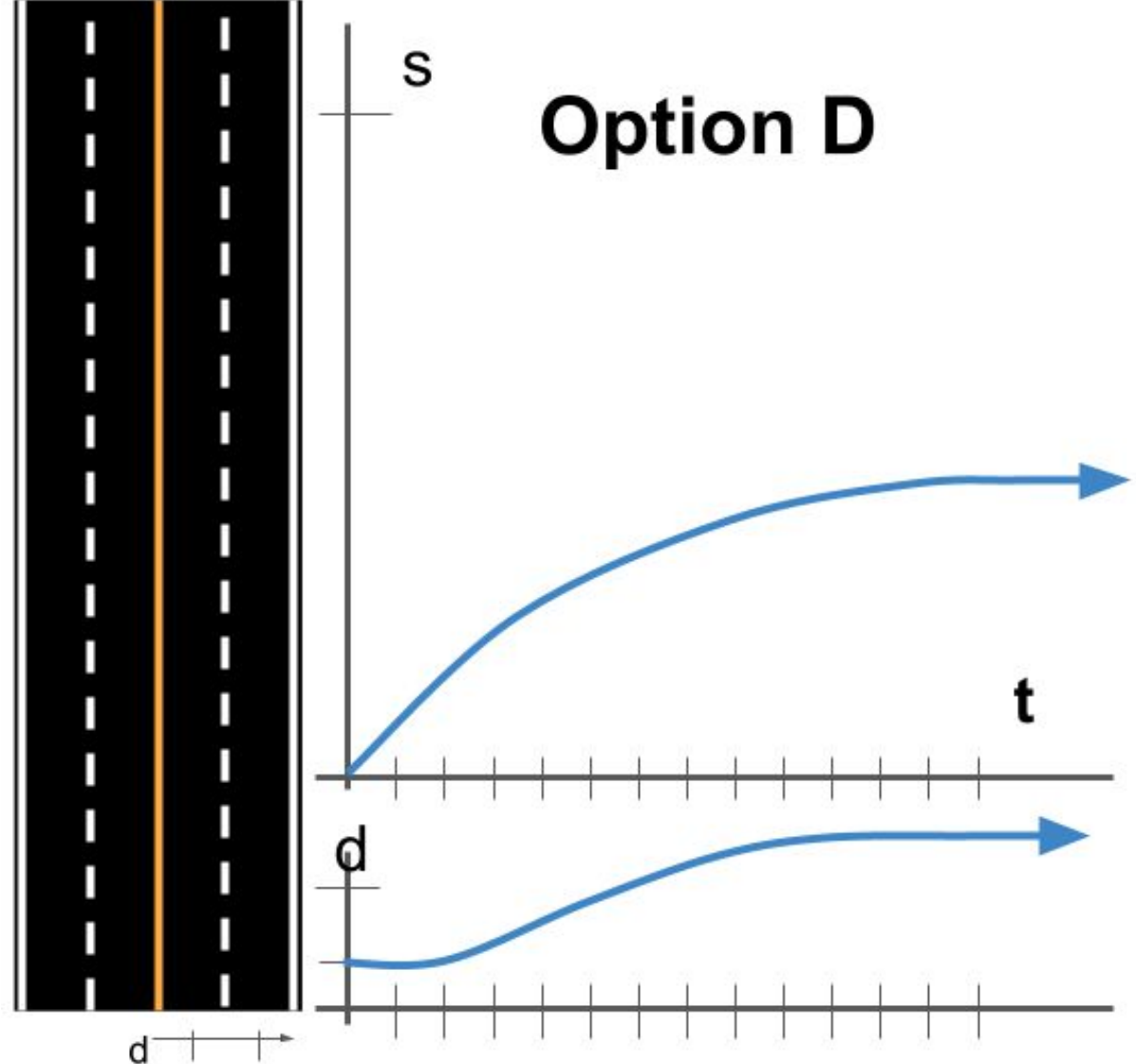
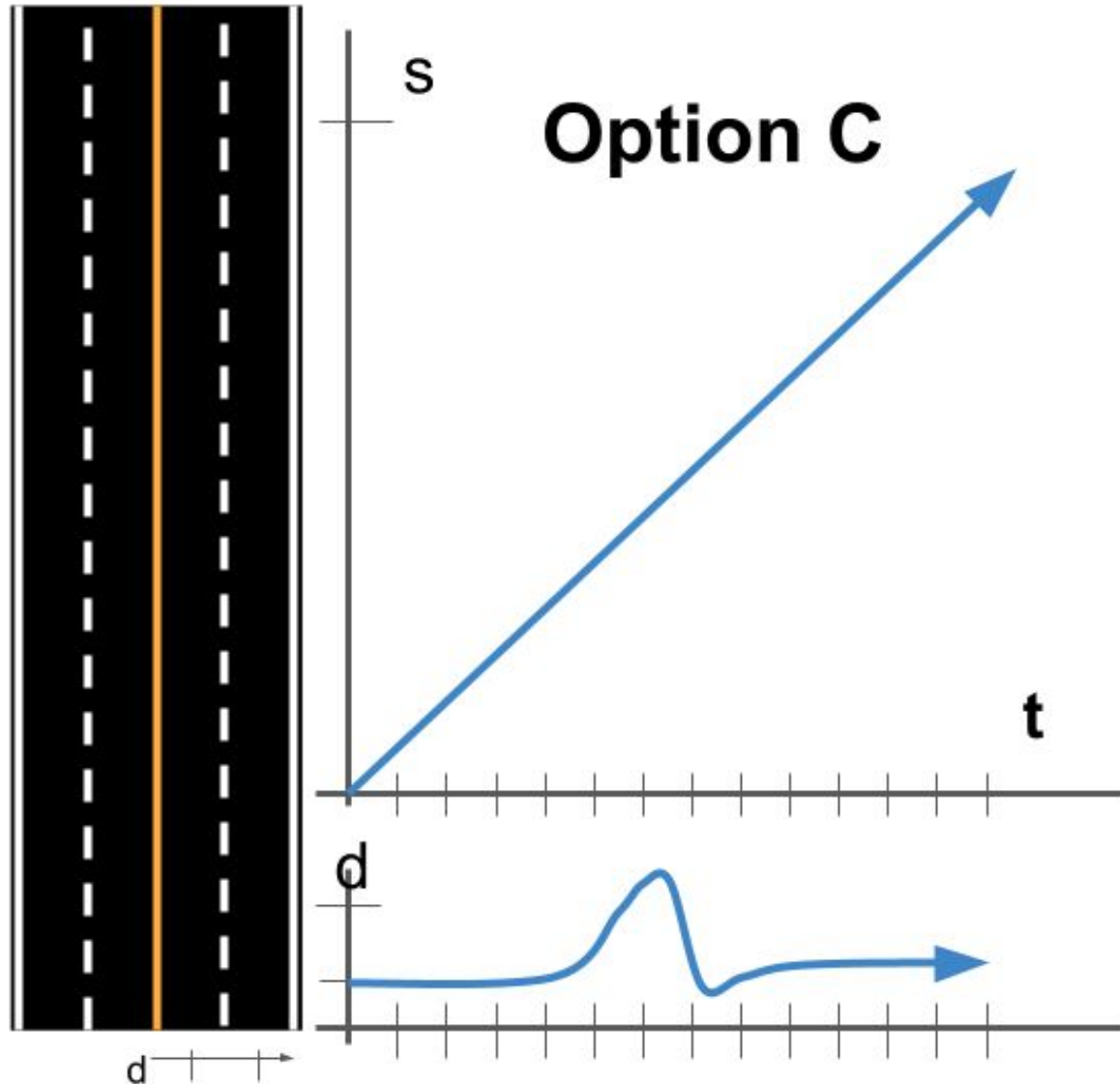
Trajectory: configurations set up in time



Trajectory Matching



Trajectory Matching



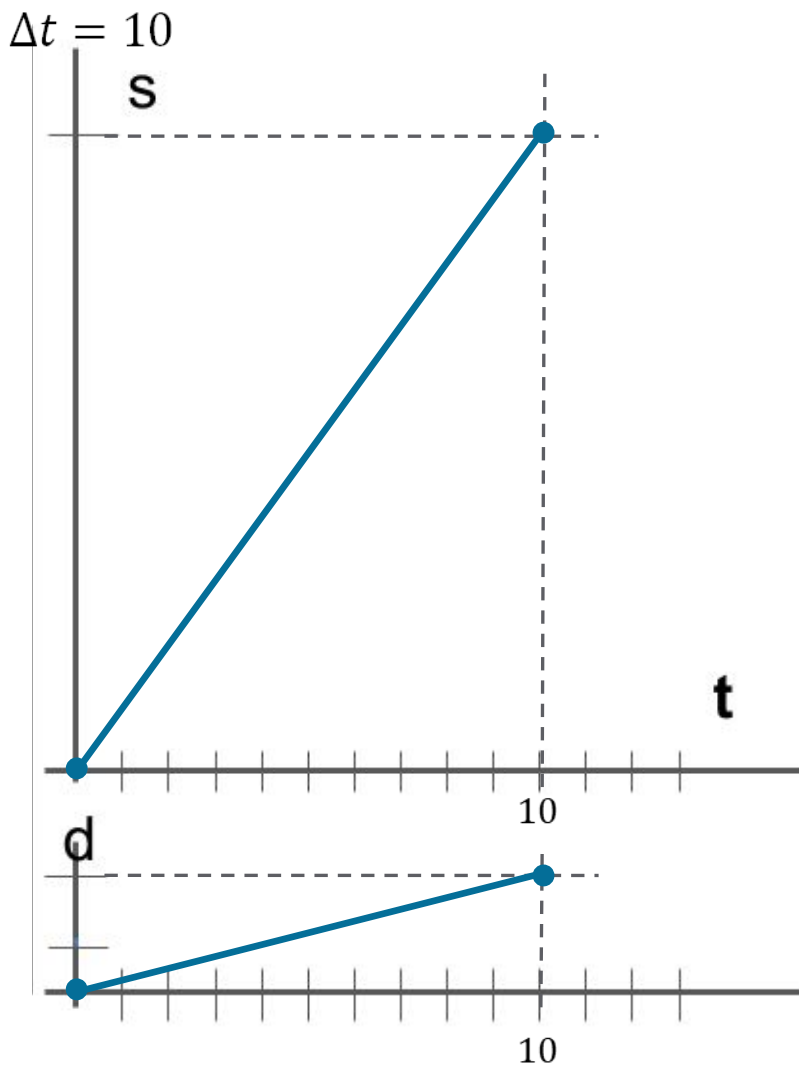
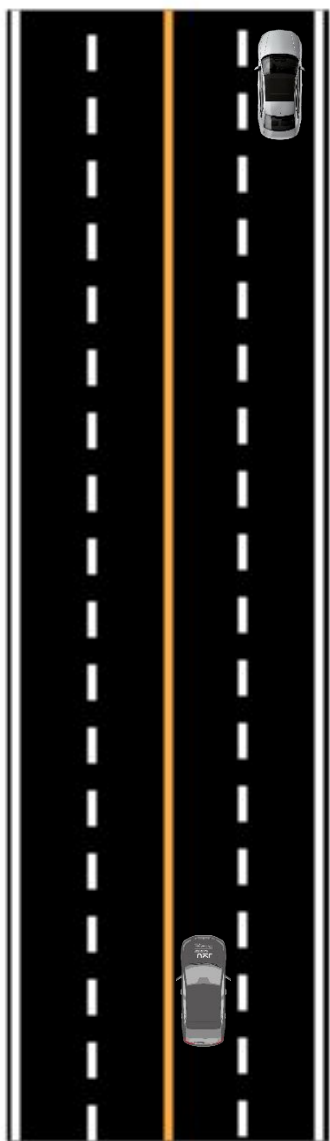
Structured Trajectory Generation Overview

- Boundary Conditions
- Jerk Minimization
- Polynomial Trajectories
- Deriving Coefficients
- Example - highway trajectory
- Evaluating feasibility and cost
- Trajectory Generation Algorithm



<https://www.manager-magazin.de/lifestyle/auto/selbststeuernde-fahrzeuge-alles-auf-autopilot-a-933093.html>

Trajectories with Boundary Conditions



Longitudinal (s) Conditions

1. S_i Initial Position
2. S_f Final Position
3. $\dot{S}_i$ Initial Velocity
4. $\dot{S}_f$ Final Velocity
5. $\ddot{S}_i$ Initial Acceleration
6. $\ddot{S}_f$ Final Acceleration

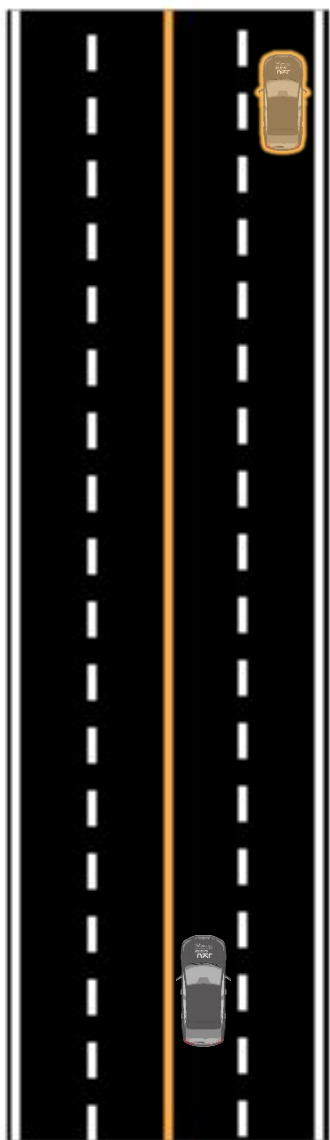
Need position continuity
Need velocity continuity

Position Velocity Acceleration Jerk Snap Crackle Pop

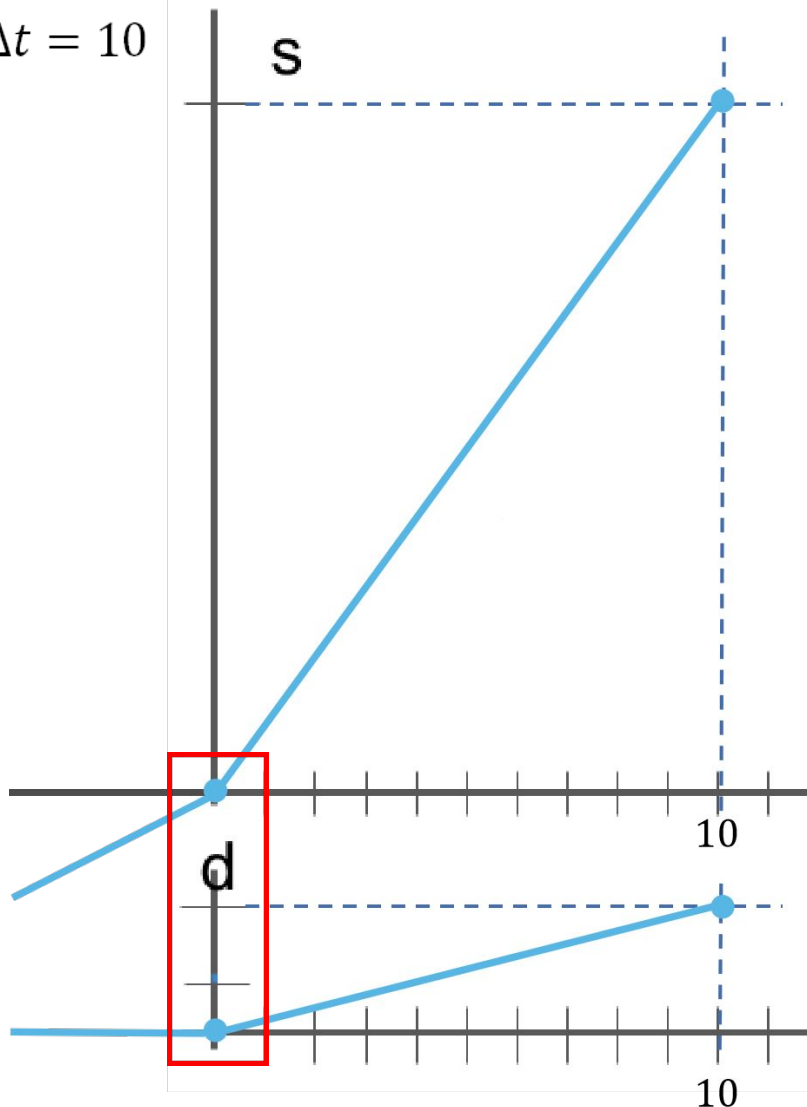
Humans feel discomfort when ? is high
Velocity? No - airplanes
Acceleration? No - freefall
Jerk? Yes - Bumper cars

=> Design for jerk minimization

Trajectories with Boundary Conditions



$$\Delta t = 10$$



Longitudinal (s) Conditions

1. s_i Initial Position
2. s_f Final Position
3. $\dot{s}_i$ Initial Velocity
4. $\dot{s}_f$ Final Velocity
5. $\ddot{s}_i$ Initial Acceleration
6. $\ddot{s}_f$ Final Acceleration

→ Need position and velocity continuity

Position → Velocity → Acceleration → Jerk → Snap → Crackle → Pop

Humans feel discomfort when ? is high

- Velocity? No - airplanes
- Acceleration? No - freefall
- Jerk? Yes - Bumper cars

Design for jerk minimization

Jerk Minimizing Trajectories

$$s(t) \in [0, t_f]$$

$$\text{Jerk} = \ddot{s}(t)$$

$$\text{Total (squared) Jerk} = \int_0^{t_f} \ddot{s}(t)^2 dt$$

Minimize this!

$$\frac{d^m s}{dt^m} = 0 \quad \forall m \geq 6$$

$$s(t) = \alpha_0 + \alpha_1 t + \alpha_2 t^2 + \dots + \alpha_n t^n + \dots$$

$$s(t) = \sum_{n=0}^{\infty} \alpha_n t^n$$

Using THIS form of s
And incorporating THIS realization

$$\alpha_6, \alpha_7, \dots = 0$$

MINIMUM 1D JERK TRAJECTORIES

$$s(t) = \alpha_0 + \alpha_1 t + \alpha_2 t^2 + \alpha_3 t^3 + \alpha_4 t^4 + \alpha_5 t^5$$

Longitudinal (s) conditions

1. S_i Initial Position
2. S_f Final Position
3. $\dot{S}_i$ Initial Velocity
4. $\dot{S}_f$ Final Velocity
5. $\ddot{S}_i$ Initial Acceleration
6. $\ddot{S}_f$ Final Acceleration

Lateral (d) conditions

1. d_i Initial Position
2. d_f Final Position
3. $\dot{d}_i$ Initial Velocity
4. $\dot{d}_f$ Final Velocity
5. $\ddot{d}_i$ Initial Acceleration
6. $\ddot{d}_f$ Final Acceleration

6 coefficients → 6 Tunable Parameters → 6 Boundary Conditions

$$[s_i, \dot{s}_i, \ddot{s}_i, s_f, \dot{s}_f, \ddot{s}_f]$$

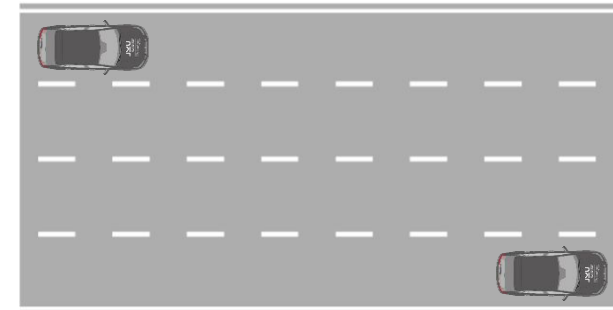
$$[d_i, \dot{d}_i, \ddot{d}_i, d_f, \dot{d}_f, \ddot{d}_f]$$

$$[0, 0, 0, 10, 0, 0]$$

INITIAL

$$s_i = 0$$

$$d_i = 0$$

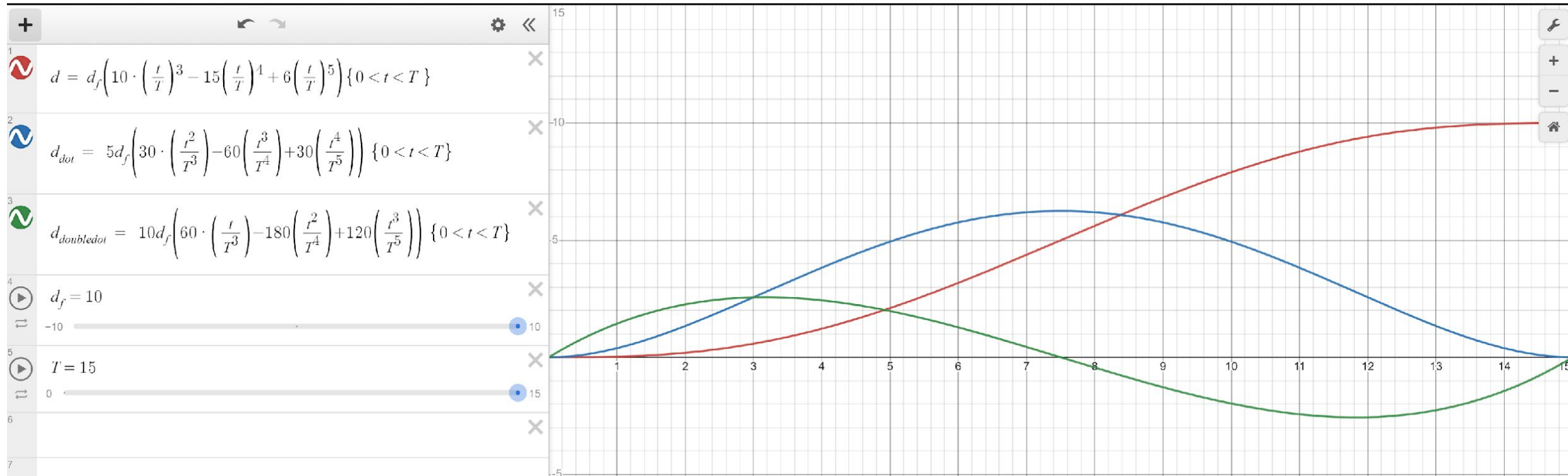


FINAL

$$s_f = 30$$

$$d_f = 10$$

Jerk Minimizing Trajectories



<https://www.desmos.com/calculator/xarnamvgum>

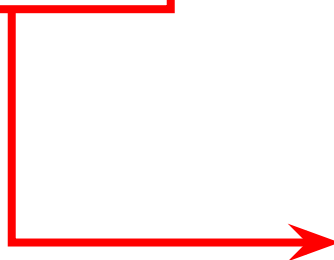
Derivation Overview

$$s(t) = \alpha_0 + \alpha_1 t + \alpha_2 t^2 + \alpha_3 t^3 + \alpha_4 t^4 + \alpha_5 t^5$$

$$\dot{s}(t) = \alpha_1 + 2\alpha_2 t + 3\alpha_3 t^2 + 4\alpha_4 t^3 + 5\alpha_5 t^4$$

$$\ddot{s}(t) = 2\alpha_2 + 6\alpha_3 t + 12\alpha_4 t^2 + 20\alpha_5 t^3$$

Choose $t_i = 0$


$$s_i = s(0) = \alpha_0$$

$$\dot{s}_i = \dot{s}(0) = \alpha_1$$

$$\ddot{s}_i = \ddot{s}(0) = 2\alpha_2$$

Longitudinal (s) conditions

1. S_i Initial Position
2. S_f Final Position
3. $\dot{S}_i$ Initial Velocity
4. $\dot{S}_f$ Final Velocity
5. $\ddot{S}_i$ Initial Acceleration
6. $\ddot{S}_f$ Final Acceleration

Derivation Overview

$$s(t) = s_i + \dot{s}_i t + \frac{\ddot{s}_i}{2} t^2 + \alpha_3 t^3 + \alpha_4 t^4 + \alpha_5 t^5$$

$$\dot{s}(t) = \dot{s}_i + \frac{\ddot{s}_i}{2} t^2 + 3\alpha_3 t^2 + 4\alpha_4 t^3 + 5\alpha_5 t^4$$

$$\ddot{s}(t) = \ddot{s}_i + 6\alpha_3 t + 12\alpha_4 t^2 + 20\alpha_5 t^3$$

$$s(t) = \alpha_3 t^3 + \alpha_4 t^4 + \alpha_5 t^5 + C_1$$

$$\dot{s}(t) = 3\alpha_3 t^2 + 4\alpha_4 t^3 + 5\alpha_5 t^4 + C_2$$

$$\ddot{s}(t) = 6\alpha_3 t + 12\alpha_4 t^2 + 20\alpha_5 t^3 + C_3$$

Longitudinal (s) conditions

1. S_i Initial Position
2. S_f Final Position
3. $\dot{S}_i$ Initial Velocity
4. $\dot{S}_f$ Final Velocity
5. $\ddot{S}_i$ Initial Acceleration
6. $\ddot{S}_f$ Final Acceleration

Derivation Overview

$$s(t) = \alpha_3 t^3 + \alpha_4 t^4 + \alpha_5 t^5 + C_1$$

$$\dot{s}(t) = 3\alpha_3 t^2 + 4\alpha_4 t^3 + 5\alpha_5 t^4 + C_2$$

$$\ddot{s}(t) = 6\alpha_3 t + 12\alpha_4 t^2 + 20\alpha_5 t^3 + C_3$$



$$s(t_f) = s_f = \alpha_3 t_f^3 + \alpha_4 t_f^4 + \alpha_5 t_f^5 + C_1$$

$$\dot{s}(t_f) = \dot{s}_f = 3\alpha_3 t_f^2 + 4\alpha_4 t_f^3 + 5\alpha_5 t_f^4 + C_2$$

$$\ddot{s}(t_f) = \ddot{s}_f = 6\alpha_3 t_f + 12\alpha_4 t_f^2 + 20\alpha_5 t_f^3 + C_3$$

Longitudinal (s) conditions

1. S_i Initial Position
2. S_f Final Position
3. $\dot{S}_i$ Initial Velocity
4. $\dot{S}_f$ Final Velocity
5. $\ddot{S}_i$ Initial Acceleration
6. $\ddot{S}_f$ Final Acceleration

Derivation Overview

We know $s_f, \dot{s}_f, \ddot{s}_f, t_f$

Known

Unknown

$$\begin{aligned} s(t_f) &= s_f = (t_f^3)\alpha_3 + (t_f^4)\alpha_4 + (t_f^5)\alpha_5 + C_1 \\ \dot{s}(t_f) &= \dot{s}_f = (3t_f^2)\alpha_3 + (4t_f^3)\alpha_4 + (5t_f^4)\alpha_5 + C_2 \\ \ddot{s}(t_f) &= \ddot{s}_f = (6t_f)\alpha_3 + (12t_f^2)\alpha_4 + (20t_f^3)\alpha_5 + C_3 \end{aligned}$$

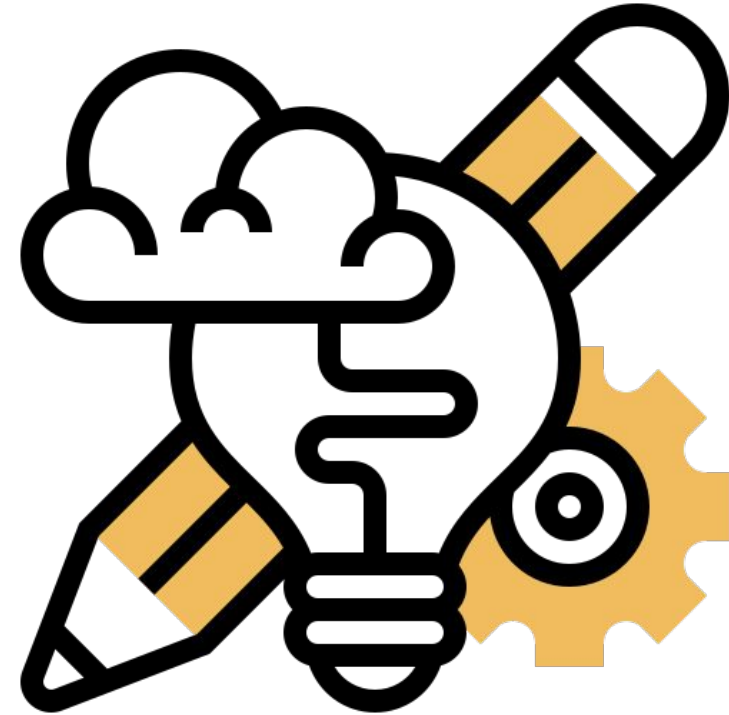

$$\begin{bmatrix} t_f^3 & t_f^4 & t_f^5 \\ 3t_f^2 & 4t_f^3 & 5t_f^4 \\ 6t_f & 12t_f^2 & 20t_f^3 \end{bmatrix} \times \begin{bmatrix} \alpha_3 \\ \alpha_4 \\ \alpha_5 \end{bmatrix} = \begin{bmatrix} s_f \\ \dot{s}_f \\ \ddot{s}_f \end{bmatrix} \begin{matrix} -C_1 \\ -C_2 \\ -C_3 \end{matrix}$$

Derivation Overview

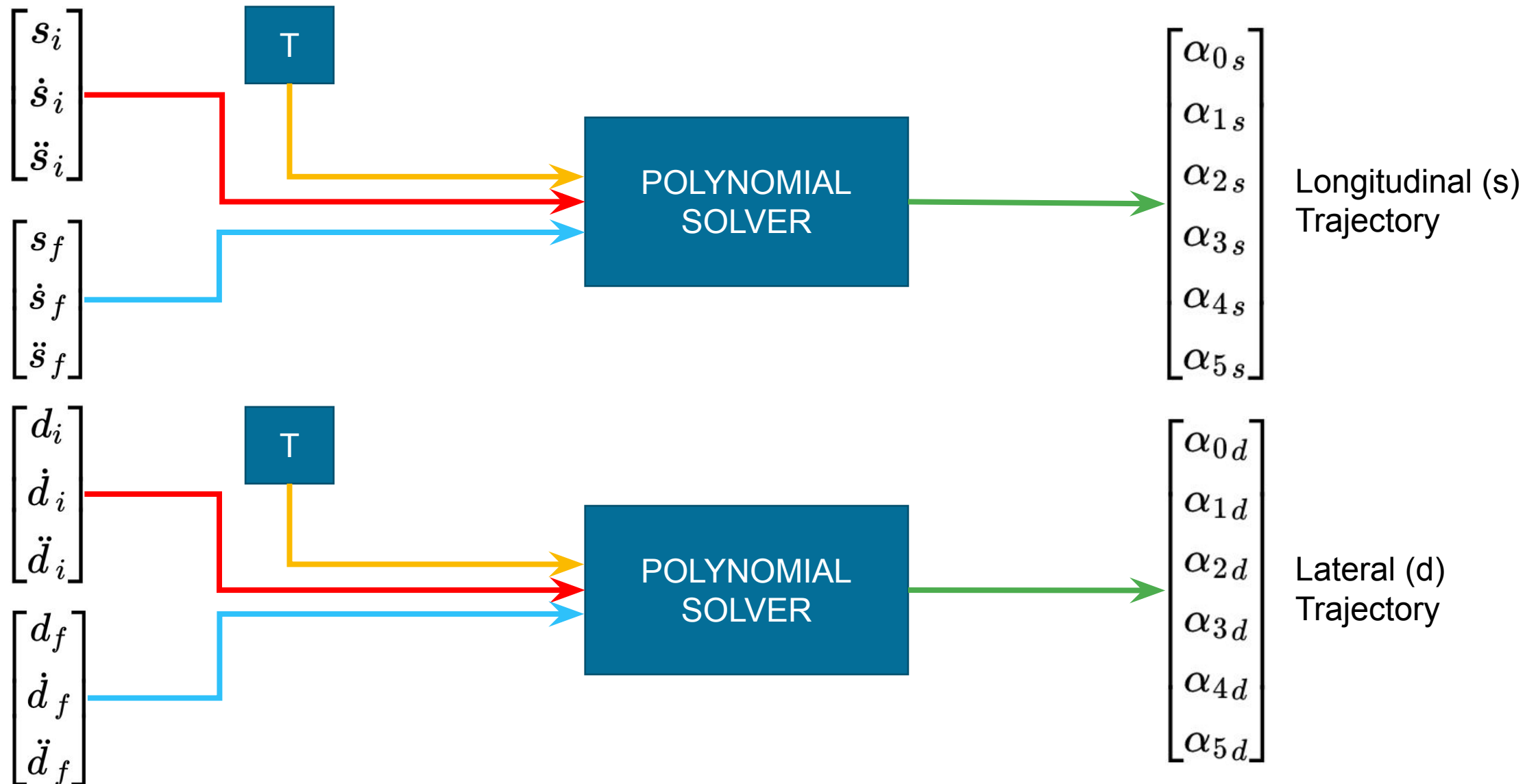
Now we have:

1. Stated that Jerk Minimizing Polynomials are degree 5 polynomials with 6 tunable parameters.
2. Given an overview of how to calculate the coefficients for such a polynomial given the boundary conditions $[s_i, \dot{s}_i, \ddot{s}_i]$ and $[s_f, \dot{s}_f, \ddot{s}_f]$ (or the corresponding d coordinates).

➤ Polynomial Trajectory Generation



Polynomial Trajectory Generation



Polynomial Trajectory Generation

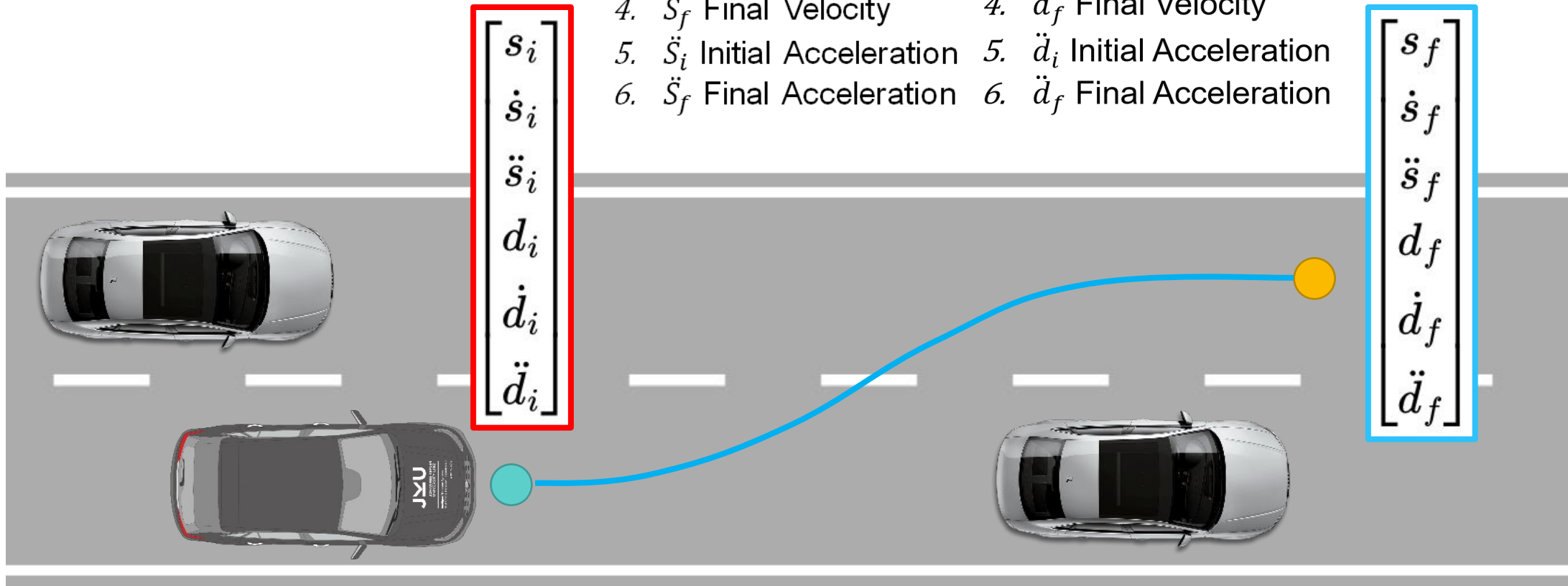
HOW POLYNOMIAL TRAJECTORY GENERATION WORKS

Longitudinal (s) conditions

1. s_i Initial Position
2. s_f Final Position
3. $\dot{s}_i$ Initial Velocity
4. $\dot{s}_f$ Final Velocity
5. $\ddot{s}_i$ Initial Acceleration
6. $\ddot{s}_f$ Final Acceleration

Lateral (d) conditions

1. d_i Initial Position
2. d_f Final Position
3. $\dot{d}_i$ Initial Velocity
4. $\dot{d}_f$ Final Velocity
5. $\ddot{d}_i$ Initial Acceleration
6. $\ddot{d}_f$ Final Acceleration



Implement a Quintic Polynomial Solver

In this exercise you will implement a quintic polynomial solver. This will let you take boundary conditions as input and generate a polynomial trajectory which matches those conditions with minimal jerk.

Inputs

Your solver will take three inputs.

1. start - $[s_i, \dot{s}_i, \ddot{s}_i]$
2. end - $[s_f, \dot{s}_f, \ddot{s}_f]$
3. T - the duration of maneuver in seconds.

Instructions

1. Implement the JMT(start, end, T) function
2. Run the script to see if you're correct!

Download the code from EX_Planning.zip in Moodle



Implement a Quintic Polynomial Solver

Tips:

Remember, you are solving a system of equations: matrices will be helpful!

The equations for position, velocity, and acceleration are given by:

$$s(t) = s_i + \dot{s}_i t + \frac{\ddot{s}_i}{2} t^2 + \alpha_3 t^3 + \alpha_4 t^4 + \alpha_5 t^5$$

$$\dot{s}(t) = \dot{s}_i + \frac{\ddot{s}_i}{2} t^2 + 3\alpha_3 t^2 + 4\alpha_4 t^3 + 5\alpha_5 t^4$$

$$\ddot{s}(t) = \ddot{s}_i + 6\alpha_3 t + 12\alpha_4 t^2 + 20\alpha_5 t^3$$

and if you evaluate these at $t=0$ you find the first three coefficients of your JMT are:

$$[\alpha_0, \alpha_1, \alpha_2] = \left[s_i, \dot{s}_i, \frac{1}{2} \ddot{s}_i \right]$$

and you can get the last three coefficients by evaluating these equations at $t=T$. When you carry out the math and write the problem in matrix form you get the following:

$$\begin{bmatrix} T^3 & T^4 & T^5 \\ 3T^2 & 4T^3 & 5T^4 \\ 6T & 12T^2 & 20T^3 \end{bmatrix} \times \begin{bmatrix} \alpha_3 \\ \alpha_4 \\ \alpha_5 \end{bmatrix} = \begin{bmatrix} s_f - \left(s_i + \dot{s}_i T + \frac{1}{2} \ddot{s}_i T^2 \right) \\ \dot{s}_f - (\dot{s}_i + \ddot{s}_i T) \\ \ddot{s}_f - \ddot{s}_i \end{bmatrix}$$

All these quantities are known except for $\alpha_3, \alpha_4, \alpha_5$

What should be checked?

When evaluating the feasibility of a potential trajectory all the following should be checked:

- Max Velocity (wrt car capabilities and speed limit)
 - Cannot exceed max speed of vehicle
 - Minimum velocity → usually shouldn't be negative (no backwards)
- Min Velocity
 - Lateral acceleration needs to be checked to avoid rollover/skid
 - Longitudinal acceleration needs to be checked with powertrain capabilities
 - Corresponds to max braking force
- Max Acceleration
 - Cannot exceed max/min steering
- Min Acceleration
- Steering Angle

Implementing Feasibility

Longitudinal Acceleration = $\dot{S}$

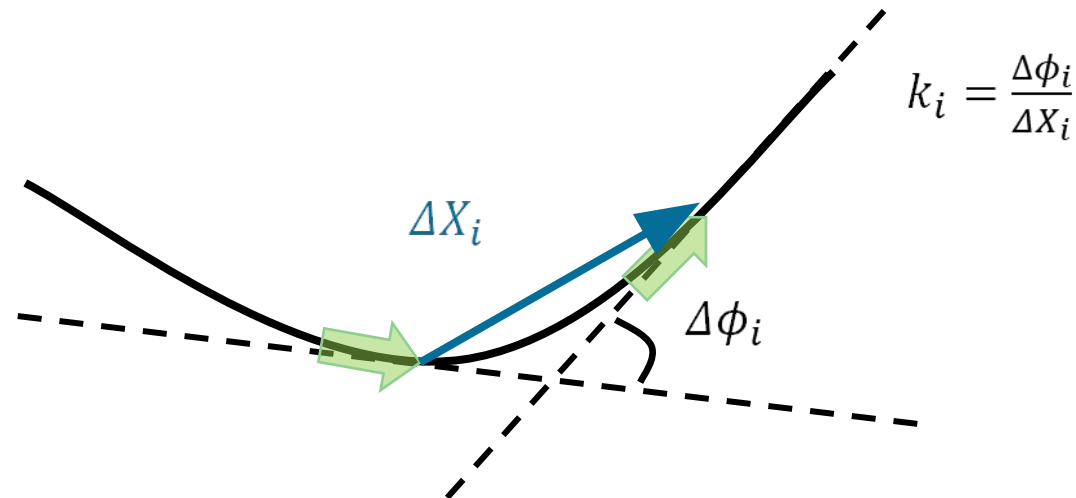
$$a_{MAX\ BREAKING} < \dot{S} < a_{MAX\ ACC}$$

Lateral Acceleration $|\ddot{d}| < a_y$

Longitudinal Speed = $\dot{S}$

$$v_{MIN} < \dot{S} < v_{SPEED\ LIMIT}$$

$$\tan \delta = \frac{L}{R} \rightarrow k = \frac{\tan \delta}{L} \rightarrow k_{MAX} = \frac{\tan(\delta_{MAX})}{L}$$

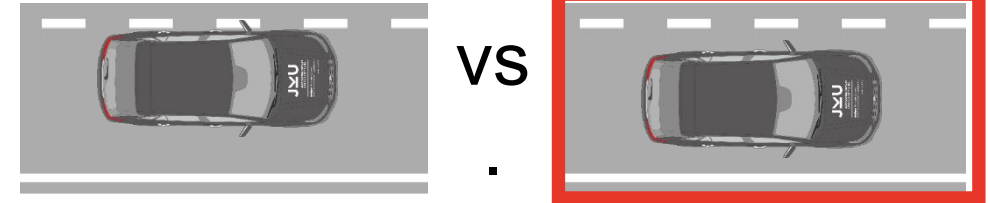


Putting it All Together

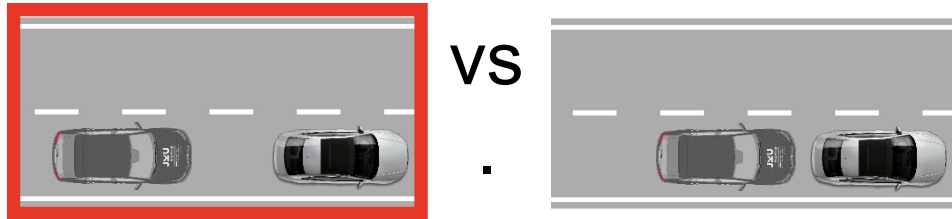
JERK (rate at which a vehicle's acceleration changes with respect to time)

$$J_d \text{ vs. } J_s$$

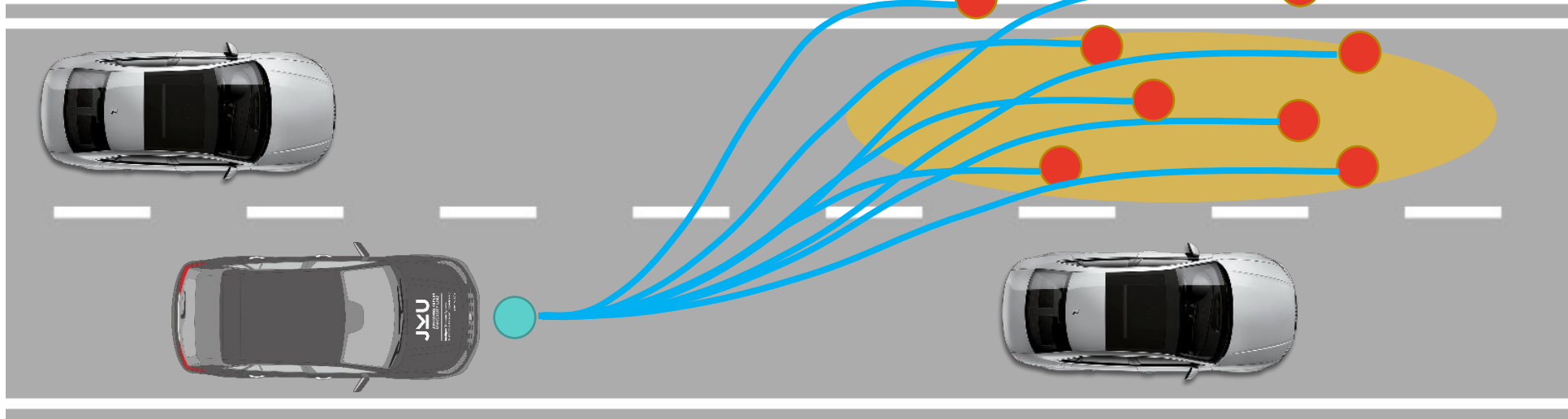
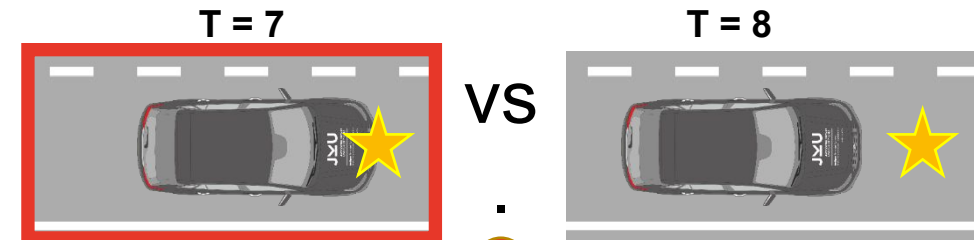
DISTANCE TO CENTER LINE



DISTANCE TO OBSTACLES



TIME TO GOAL



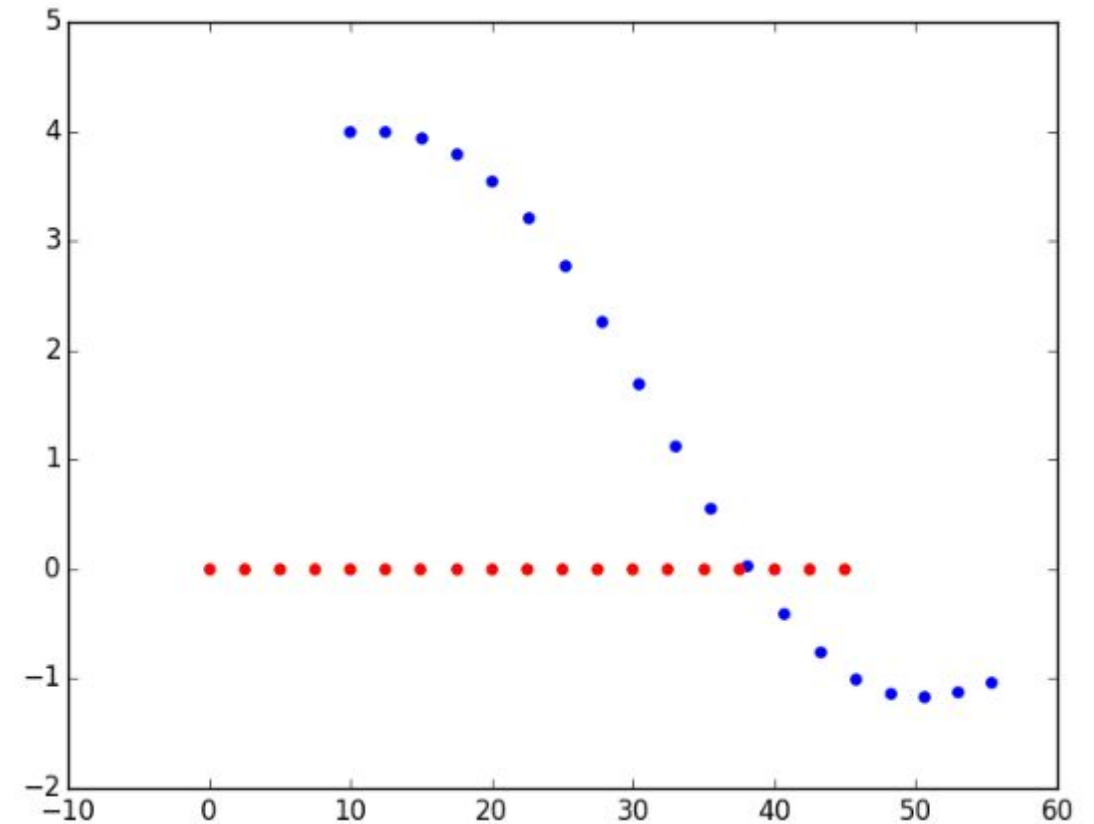
Polynomial Trajectory Generation Playground

In the Python code you will be provided, you will have a working Polynomial Trajectory Generator. You can see it in action by doing the following:

Getting Started

- Download the **project code**: TrajectoryExercise2 Part 2 EX_Planning.zip
- From the project's directory, run *python evaluate_ptg.py*. You should see a **plot similar to the one on the right**. This plot shows the **s (x-axis)** and **d(y-axis) trajectories** in the traffic from a manual driven vehicle (red) and a self-driving car (blue).

In this situation, the self-driving car was trying to get behind the target vehicle, but the cost functions it was using weren't weighted appropriately and so it didn't behave as expected.



Polynomial Trajectory Generation Playground

There are 5 files in the provided code. You'll probably want to start by modifying cost function weights in *ptg.py* but may also want to add cost functions of your own.

File Descriptions

- *ptg.py* - The primary code for generating a polynomial trajectory for some constraints. This is also where weights are assigned to cost functions. Adjusting these weights (and possibly adding new cost functions), can have a big effect on vehicle behavior.
- *cost_functions.py* - This file contains many cost functions which are used in *ptg.py* when selecting the best trajectory. Some cost functions aren't yet implemented...
- *evaluate_ptg.py* - This file sets a start state, goal, and traffic conditions and runs the PTG code. Feel free to modify the goal, add traffic, etc... to test your vehicle's trajectory generation ability.
- *constants.py* - constants like speed limit, vehicle size, etc...
- *helpers.py* - helper functions used by other files.

Optional Reading

Supporting Materials

Optimal Trajectory Generation For Dynamic Street Scenarios In A Frenet Frame

<https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=5509799>

Intention-Net: Integrating Planning and Deep Learning for Goal-Directed Autonomous Navigation

<https://arxiv.org/pdf/1710.05627.pdf>

Learning to Navigate in Cities Without a Map

<https://arxiv.org/pdf/1804.00168.pdf>

A Look at Motion Planning for Autonomous Vehicles at an Intersection

<https://arxiv.org/pdf/1806.07834.pdf>

DeepTraffic: Crowdsourced Hyperparameter Tuning of Deep Reinforcement Learning Systems for Multi-Agent Dense Traffic Navigation

<https://arxiv.org/pdf/1801.02805.pdf>